Metodología de los Sistemas II

Recuperatorio segundo Trabajo práctico

Docente: Edith Rocio Castillo Yauri
Div.: 141
Grupo N°: 6
Integrantes: Alderete Lautaro, Barrios Alfonzo Ramiro, Basso Santiago, Pokoik Lucia Laura, Cortes Malena Nahir, Anahí González Pineda, Martin Gomez Valle

Consigna:

Recuperatorio:

- Corregir **sólo** lo marcado en la devolución del TP, para ello se disponibilizará una tarea en el campus para que puedan subir el archivo.
- Rendir el cuestionario.

Correcciones marcadas en el TP:

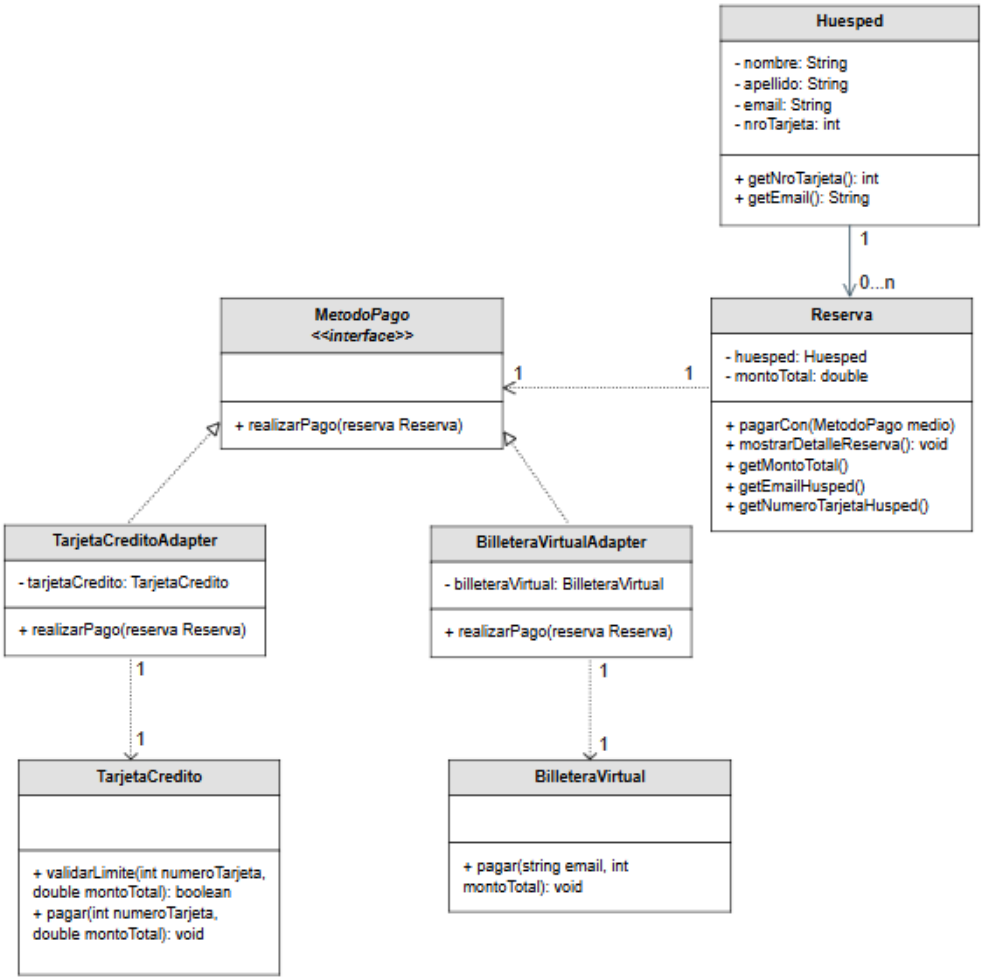
Adapter: No hace falta la relación de asociación entre Reserva y MetodoPago. El método pagar() recibe como atributo el método de pago

Observer: Sensor debe ser una generalización. No definen cardinalidad.

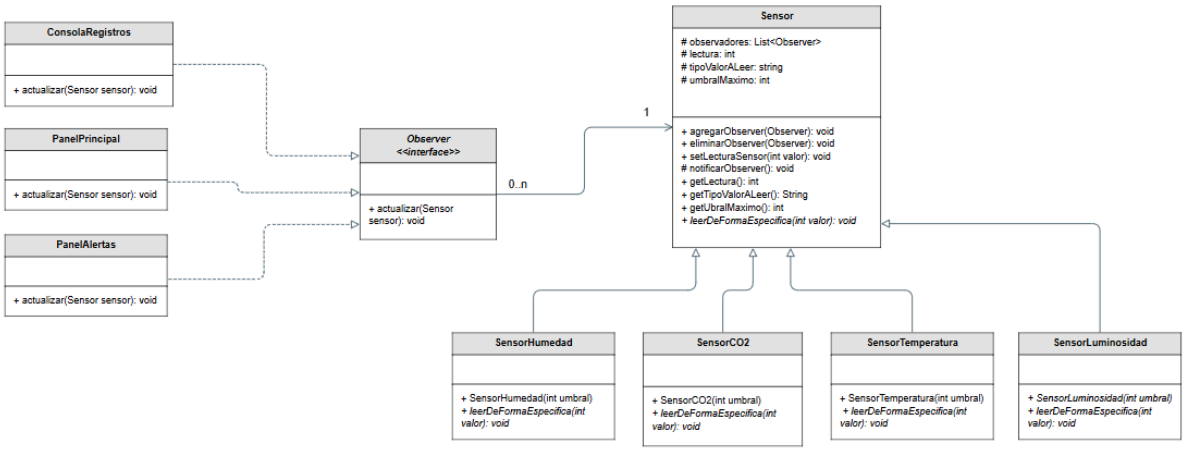
Refactor, bien encaminado. El número hardcodeado 2 en calcular daño, definir como constante.

Repositorio :
https://github.com/CortesMalena/Recuperatorio_Segundo_Trabajo_Practico_MetodologiaII

Adapter:

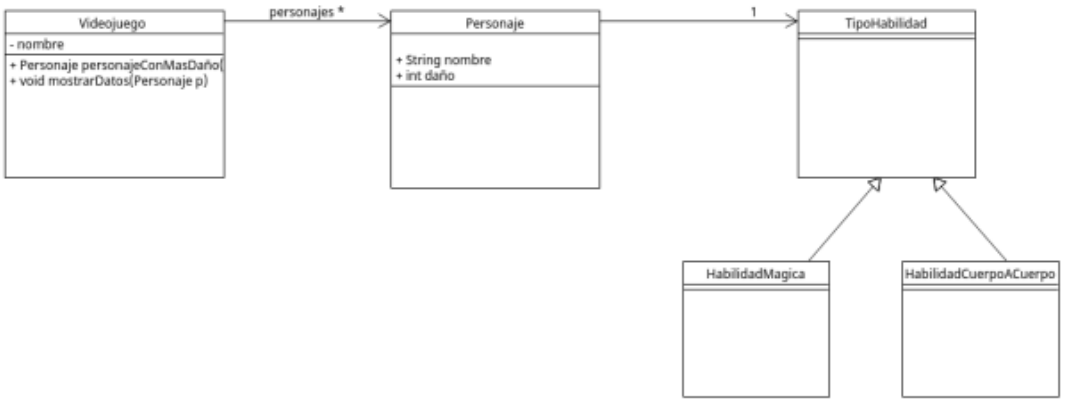

Huesped
- nombre: String
- apellido: String
- email: String
- nroTarjeta: int
+ getNroTarjeta(): int
+ getEmail(): String
1
0...n
MetodoPago
<<interface>>
+ realizarPago(reserva Reserva)
1
1
Reserva
- huesped: Huesped
- montoTotal: double
+ pagarCon(MetodoPago medio)
+ mostrarDetalleReserva(): void
+ getMontoTotal()
+ getEmailHusped()
+ getNumeroTarjetaHusped()
TarjetaCreditoAdapter
- tarjetaCredito: TarjetaCredito
+ realizarPago(reserva Reserva)
BilleteraVirtualAdapter
- billeteraVirtual: BilleteraVirtual
+ realizarPago(reserva Reserva)
1
1
1
1
TarjetaCredito
+ validarLimite(int numeroTarjeta, double montoTotal): boolean
+ pagar(int numeroTarjeta, double montoTotal): void
BilleteraVirtual
+ pagar(string email, int montoTotal): void

Observer:


ConsolaRegistros
+ actualizar(Sensor sensor): void
PanelPrincipal
+ actualizar(Sensor sensor): void
PanelAlertas
+ actualizar(Sensor sensor): void
Observer
<<interface>>
+ actualizar(Sensor sensor): void
0..n
1
Sensor
observadores: List<Observer>
lectura: int
tipoValorALeer: string
umbralMaximo: int
+ agregarObserver(Observer): void
+ eliminarObserver(Observer): void
+ setLecturaSensor(int valor): void
# notificarObserver(): void
+ getLectura(): int
+ getTipoValorALeer(): String
+ getUbralMaximo(): int
+ leerDeFormaEspecifica(int valor): void
SensorHumedad
+ SensorHumedad(int umbral)
+ leerDeFormaEspecifica(int valor): void
SensorCO2
+ SensorCO2(int umbral)
+ leerDeFormaEspecifica(int valor): void
SensorTemperatura
+ SensorTemperatura(int umbral)
+ leerDeFormaEspecifica(int valor): void
SensorLuminosidad
+ SensorLuminosidad(int umbral)
+ leerDeFormaEspecifica(int valor): void

Refactoring:

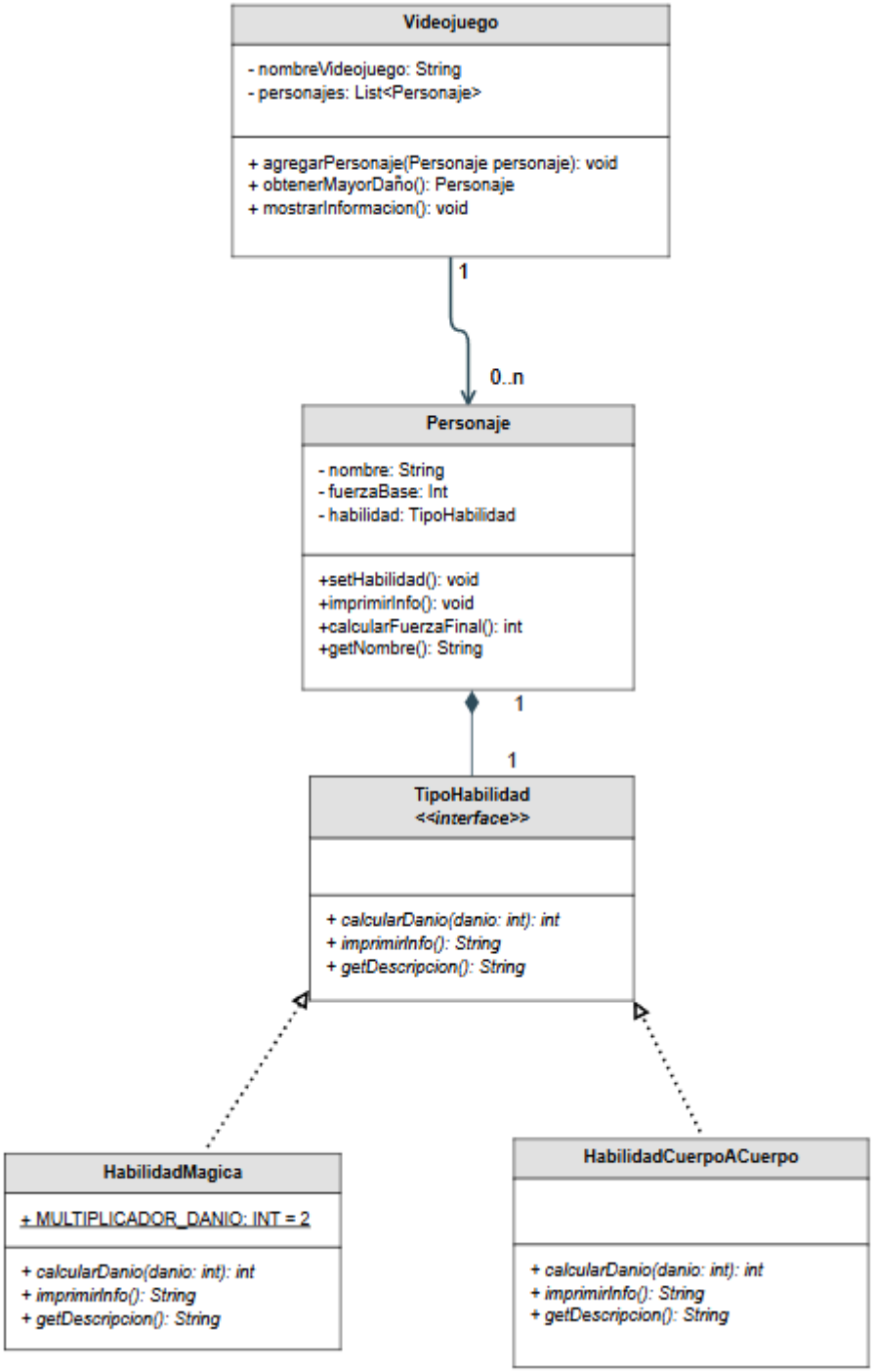
Código de la consigna:




```
public class Videojuego{

        public Personaje personaje ConMasDaño() {

        Personaje temp = null;

        double max= 0;

            for (Personaje p: personajes) {
                double daño p.getTipoHabilidad().calcularDaño (p.getDaño());
                if (daño > max) {
                    temp p;
                    max daño;
                }
            }
             return temp;
        }

        public void imprimirInfo(Personaje p) {

            System.out.println(p.getNombre() + "tiene como daño " + p.getDaño());

            if (p.getTipoHabilidad().getClass() == HabilidadMagica.class){
                System.out.println("Habilidad tipo habilidad mágica");
                System.out.println("Esta habilidad dobla tu fuerza");

            } else{
                System.out.println("Habilidad tipo habilidad Cuerpo a cuerpo");
                System.out.println("Este habilidad mantiene tu fuerza");
            }

}
```

Refactoring aplicado:

Diagrama: (4)



1) **Mal olor encontrado:**
   El problema principal del código es que usa muchos condicionales para saber que tipo de habilidad tiene un personaje:

ej: if (p.getTipoHabilidad().getClass() == HabilidadMagica.class)

Esto se llama **Switch statements** según Martin Fowler.

Además, en el método *imprimirInfo()* se realizan múltiples consultas al personaje y a su habilidad para decidir qué imprimir, en lugar de delegar ese comportamiento a las clases correspondientes. Esto genera **duplicación de lógica** y viola el principio de **Tell, Don't Ask**.

Otro mal olor es que la clase Personaje tiene solo datos y getter, no hace nada propio.

Además, se usan muchos Strings sueltos en vez de tenerlos organizados dentro de métodos o clases.

Asimismo, también se encuentran **variables temporales innecesarias** como**:**

Personaje temp = null; double max= 0;

2) Estos malos olores son un **problema** porque si agrego otro tipo de habilidad tengo que volver a modificar el if, perjudicando la escalabilidad del sistema, además, no usa polimorfismo, hay duplicación de lógica y Videojuego termina conociendo detalles internos de todas las habilidades, lo que indica que no están correctamente encapsuladas y se genera alto acoplamiento.
En otras palabras, el videojuego depende de detalles que deberían estar ocultos dentro de cada habilidad.

3) La mejor solución sería aplicar el refactoring **Replace Conditional Logic with Strategy**.
Este refactoring sirve para cambiar los if que comparan clases y reemplazarlos por polimorfismo. De esta forma, cada tipo de habilidad se encargará de su propio comportamiento y no hace falta hacer un if para saber qué hacer con cada una, implementando una interfaz común que es la estrategia siguiendo el patrón Strategy.

Además, se pueden aplicar refactorings complementarios como **Move Method,** mover la lógica de impresión y cálculo desde Videojuego hacia las clases concretas de habilidades, que son quienes realmente deben conocer ese comportamiento. y **Extract Method** (separar lógica repetida). Esto permite que el diseño quede más claro, con menos duplicación y siguiendo los principios de orientación a objetos. Asimismo, **Replace Temp With Query** que eliminaría variables temporales como temp y max cuando solo almacenan resultados intermedios, reemplazándolas por métodos que expresen mejor la intención.